

Rapport Final

Pipeline ML sécurisé — Prédiction du risque de crédit

Table des matières

1	Introduction et périmètre du projet	1
1.1	Objectifs poursuivis	1
2	Description du dataset et classification de sensibilité	1
3	Modèle de menace et diagramme de flux de données	2
3.1	Acteurs et surfaces d'attaque considérés	2
3.2	Diagramme de flux de données (Data Flow)	2
4	Implémentation du pipeline de données sécurisé	2
5	Modèle de base et conception de l'API	3
5.1	Modèle	3
5.2	API (conception initiale, Phase A)	3
6	Tests Red Team et résultats	3
7	Mitigations Blue Team et comparaison avant/après	4
8	MLOps sécurisé et artefacts de gouvernance	5
9	Risques résiduels, limites et améliorations futures	5
9.1	Risques résiduels et limites	5
9.2	Améliorations futures proposées	5
10	Conclusion	6

1 Introduction et périmètre du projet

Ce projet couvre l'ensemble du cycle de vie d'un système de machine learning destiné à prédire le risque de crédit d'un client (*Good Credit / Bad Credit*), depuis la préparation sécurisée des données jusqu'au déploiement d'une API protégée, en passant par une phase explicite d'attaque (Red Team) et de correction (Blue Team).

1.1 Objectifs poursuivis

- Construire un pipeline de données respectueux de la vie privée (suppression d'identifiants, anonymisation, pseudonymisation).
- Entraîner un modèle de classification binaire (Random Forest) et l'exposer via une API FastAPI.
- Identifier concrètement les vulnérabilités de l'API par des tests d'attaque réels.
- Corriger ces vulnérabilités et démontrer l'efficacité des corrections par des tests automatisés.
- Conteneuriser l'application (Docker) et produire les artefacts de gouvernance attendus (Model Card, Datasheet, Risk Register, etc.).

Le modèle et l'API obtenus restent des prototypes pédagogiques : les limites de performance et de sécurité sont documentées explicitement tout au long de ce rapport plutôt que dissimulées.

2 Description du dataset et classification de sensibilité

Le dataset source (`data_credit_risk.csv`) contient 3 200 clients et 25 colonnes brutes, sans valeur manquante. La variable cible est `credit_risk` (*good/bad*), un problème de classification binaire.

TABLE 1 – Classification des colonnes par catégorie de sensibilité

Catégorie	Nombre	Exemples de colonnes
Identifiants directs	5	customer_id, full_name, email, phone, national_id
Quasi-identifiants	7	age, gender, region, marital_status, education_level
Données financières sensibles	7	annual_income, savings_balance, loan_amount, debt_ratio

Les identifiants directs identifient sans ambiguïté un individu et ont été supprimés avant tout entraînement. Les quasi-identifiants, bien que non identifiants isolément, présentent un risque de réidentification par combinaison et ont été pseudonymisés (âge en tranches, revenu en quartiles). Les données financières ont été anonymisées par normalisation statistique (`StandardScaler`). Le détail complet colonne par colonne figure dans `docs/dataset_datasheet.md`.

3 Modèle de menace et diagramme de flux de données

3.1 Acteurs et surfaces d'attaque considérés

- **Attaquant externe non authentifié** : tente d'utiliser l'API sans autorisation, ou de l'inonder de requêtes.
- **Attaquant authentifié malveillant** : possède une clé API valide mais tente d'extraire le modèle ou de sonder ses frontières de décision.
- **Attaquant passif** : intercepte le trafic réseau (pertinent en l'absence de HTTPS, voir section 9).
- **Erreur d'intégrité** : altération accidentelle ou malveillante du fichier `model.pkl`.

3.2 Diagramme de flux de données (Data Flow)

```
Dataset brut (data_credit_risk.csv)
|
v
Nettoyage + identification des donnees sensibles
|
v
Suppression identifiants / anonymisation / pseudonymisation
|
v
Dataset ML-ready (credit_processed.csv) --> hash SHA-256
|
v
Entrainement du modele (Random Forest)
|
v
Modele (model.pkl) --> hash SHA-256 (model_hash.txt)
|
v
API FastAPI (src/api.py)
|-- Cle API (authentification)
|-- Validation Pydantic (bornes de valeurs)
|-- Rate limiting (100 req/min)
|-- Logs (date, heure, requete, resultat, erreur)
|-- Reponse : classe + confiance qualitative uniquement
|
v
Client autorise (analyste credit / systeme interne)
```

4 Implémentation du pipeline de données sécurisé

- **Nettoyage** : détection de valeurs aberrantes par méthode IQR élargie, remplacement par la médiane.
- **Suppression** des 5 identifiants directs avant toute étape d'entraînement.

- **Anonymisation** des 4 variables financières principales par **StandardScaler**.
- **Pseudonymisation** des quasi-identifiants (âge en 5 tranches, revenu en 4 quartiles).
- **Masquage** : remplacement de l'identifiant client par une référence générique (USER0001...).
- **Encodage** : **Label Encoding** pour les variables binaires et la cible, **One-Hot Encoding** pour les variables multi-catégories.
- **Traçabilité** : hash SHA-256 calculé sur le dataset brut et sur le dataset traité (dossier `hashes/`).

Dataset final : `data/processed/credit_processed.csv`, 3 200 lignes, 41 colonnes (37 features numériques/booléennes utilisées pour l'entraînement après retrait des colonnes non numériques résiduelles).

5 Modèle de base et conception de l'API

5.1 Modèle

TABLE 2 – Performances du modèle

Métrique	Test Set	Train Set
Accuracy	67,81 %	91,52 %
Precision	65,92 %	91,36 %
Recall	60,48 %	89,87 %
F1-score	63,08 %	90,61 %

Algorithme : *Random Forest Classifier* (scikit-learn), 100 arbres, profondeur maximale 10, split 80/20 stratifié. L'écart de 23,71 points entre Train et Test révèle un sur-apprentissage significatif (détaillé en section 9).

5.2 API (conception initiale, Phase A)

Une API FastAPI expose un unique endpoint `POST /predict` qui charge le modèle au démarrage et retourne la classe prédite ainsi que le score de confiance exact et la distribution complète des probabilités — une conception volontairement vulnérable, utilisée comme baseline pour la phase Red Team.

6 Tests Red Team et résultats

Cinq scripts d'attaque (dossier `attacks/`) ont été exécutés contre l'API baseline pour révéler ses vulnérabilités.

TABLE 3 – Résultats des tests Red Team

Test	Résultat obtenu	Vulnérabilité
Entrées malformées Valeurs négatives/extrêmes	Fuite de confiance acceptées (code 200)	Absence de validation de plage
Frontière de décision	localisée en 12 requêtes (<code>debt_ratio > 0.3126</code>)	Score de confiance exposé
Abus de requêtes	100 requêtes parallèles acceptées, aucun code 429	Absence de rate limiting
Extraction de modèle	Clone fidèle à 83,3% avec 400 requêtes	Modèle clonable via l'API
Boundary probing	<code>past_due_count</code> : plus d'incidents améliore le score (illogique)	Comportement instable du modèle

Détail complet des attaques, scripts et résultats dans `docs/attack_report.md`.

7 Mitigations Blue Team et comparaison avant/après

TABLE 4 – Comparaison Phase A vs Phase C

Aspect	Avant (Phase A)	Après (Phase C)
Authentification	Aucune	Clé API obligatoire (401 sinon)
Confiance	Score exact exposé (52,15%)	Niveau qualitatif (<i>Faible/Moyenne/Élevée</i>)
Validation	Type seulement	Bornes de valeurs + champs interdits (<code>extra=forbid</code>)
Logs	Aucun	<code>logs/api.log</code> (date, heure, requête, résultat, erreur)
Limitation de requêtes	Aucune (100/100 acceptées)	100/minute (429 au-delà)
Messages d'erreur	Détails techniques exposés	Message générique (<i>Entrée invalide.</i>)

Chaque protection a été validée par une suite de 12 tests automatisés (`tests/test_api_security.py` exécutée avec `pytest`), tous réussis. Détail dans `docs/before_after_metrics.md` et `docs/mitigation_report.md`.

8 MLOps sécurisé et artefacts de gouvernance

- **Conteneurisation** : Dockerfile (image python:3.11-slim) + docker-compose.yml, lancement en une commande (docker compose up).
- **Intégrité du modèle** : hash SHA-256 calculé et documenté (models/model_hash.txt).
- **Model Card** (docs/model_card.md) : identité, objectif, utilisateurs, données, performances, limites, risques de sécurité.
- **Dataset Datasheet** (docs/dataset_datasheet.md) : origine, colonnes, données sensibles, nettoyage, transformations, risques de confidentialité, consentement, conservation.
- **Risk Register** (docs/risk_register.md) : 18 risques recensés, avec probabilité, impact et statut de traitement.
- **Security Checklist** (docs/security_checklist.md) : 29 points de contrôle, 22 validés (~76%).
- **Cycle de vie du modèle** documenté de bout en bout (docs/model_lifecycle.md).

9 Risques résiduels, limites et améliorations futures

9.1 Risques résiduels et limites

- Dataset de taille limitée (3 200 lignes), contribuant au sur-apprentissage observé.
- Comportement contre-intuitif du modèle sur `past_due_count` (signal de sur-apprentissage / corrélation fallacieuse).
- Clé API unique et statique, partagée entre tous les utilisateurs (pas de gestion par utilisateur).
- Absence de HTTPS/TLS (trafic en clair en environnement de test local).
- Consentement (`consent_flag`) non vérifié avant l'entraînement.
- Secrets encore codés en dur (`docker-compose.yml`) plutôt que dans un fichier `.env` dédié.
- Hash du modèle disponible mais non vérifié automatiquement au démarrage de l'API.
- Absence de surveillance / alerting en temps réel au-delà des logs bruts.

9.2 Améliorations futures proposées

- Ré-entraînement avec régularisation renforcée et validation croisée pour réduire le sur-apprentissage.
- Migration vers une authentification forte (OAuth2 / JWT) avec gestion de rôles.
- Mise en place de HTTPS via un reverse proxy en environnement de déploiement réel.
- Vérification automatique du hash du modèle au démarrage de l'API.
- Surveillance en temps réel (ex. Prometheus/Grafana) avec alerting sur comportements suspects.

- Remplacement du CSV statique par une base de données, avec pipeline CI/CD pour automatiser ré-entraînement et déploiement.
- Audit de biais formel du modèle par sous-groupe démographique.

Détail complet dans `docs/risks_and_improvements.md`.

10 Conclusion

Ce projet a permis de construire, de bout en bout, un pipeline de machine learning sensibilisé à la sécurité et à la protection des données personnelles : préparation respectueuse de la vie privée, modèle entraîné et documenté honnêtement (y compris ses limites), API volontairement mise à l'épreuve par une phase d'attaque réelle, puis sécurisée méthodiquement en réponse à chaque vulnérabilité identifiée, et enfin conteneurisée avec les artefacts de gouvernance attendus dans un contexte professionnel (Model Card, Datasheet, Risk Register, Security Checklist).

La démarche Red Team / Blue Team a démontré concrètement l'intérêt de challenger un système avant de le protéger : chaque protection ajoutée répond à une attaque réellement exécutée et documentée, plutôt qu'à une liste théorique de bonnes pratiques. Les limites restantes — modèle perfectible, authentification à renforcer, surveillance à mettre en place — sont explicitement assumées comme un point de départ pour une itération future, plutôt que dissimulées.